```
def deco(func):
  @wraps(func)
  def wrapper(*args, **kwargs):
    SQLCounter.clear_data()
    SQLCounter.current_func(func)
    start_time = datetime.now()
    result = func(*args, **kwargs)
    timedelta = datetime.now() - start_time
    print 'Function: {}.{}'.format(
        func.__module__, func.func_name)
    print 'Total processing time: {} ms'.format(
        timedelta.total_seconds()*1000)
    SQLCounter.show_data()
    return result
  return wrapper

class goofy(object):
  @staticmethod
  def profile():
    return goofy_profiler

  @staticmethod
  def call_handler(func, lineno, colno,  *args, **kwargs):
    sargs = (" {}(..)".format(func.__name__), lineno, colno)
    SQLCounter.before(*sargs)
    result = func(*args, **kwargs)
    SQLCounter.after(*sargs)
    return result
```

Also we'd like to patch the execute_sql method of SQLCompiler to collect sql queries, the code for which you can find here – http://dgit.in/exctSQL

And then there's the code of SQLCounter which collects SQL queries and also the information of what SQL queries get executed at different position inside a function. Refer to http://dgit.in/SQLCounter for the code.

Usage

There is a view `get_bot_submission_response` in our codebase and we applied `goofy.profile()` decorator on it to profile it.

```
from goofy import goofy
@goofy.profile()
def get_bot_submission_response(request, game):
  ...
```

When the view gets called it prints following output to console.

```
Function: problems.views.get_bot_submission_response
Total processing time: 201.499 ms
Total SQL queries: 8,  Total time: 9.856 ms
```

Location	Hit	Queries	Time (ms)	Avg Time (ms)
line no 100 : .user	1	1	0.97	0.97
line no 100 : .player1	1	1	1.38	1.38
line no 112 : .problem	1	1	1.52	1.52
line no 119 : player_2_name(..)	1	2	2.193	1.0965
line no 138 : get_game_data(..)	1	2	2.244	1.122
line no 163 : render_to_string(..)	1	1	1.549	1.549

All the `.<attribute>` tells the location of attribute access which triggered SQL queries. `.user`, `.player1`, `.problem` are attribute access and `player_2_name(..)`, `get_game_data(..)`, `render_to_string(..)` are function calls. We've used the tabulate package to pretty print the table.

What's next

Not only SQL but any type of queries can be profiled in this model. We also added the feature to profile memcached queries in goofy profiler. Lots of more improvements can be done upon it and we're working on that. Be careful about the syntax for the code in this article, and also keep a backup before altering your existing code.

We'll soon clean the goofy profiler's code and open-source it on github. And that means pretty soon you'll be able to profile SQL queries from your Django code. This tracking will help you find out segments of the code that need optimization. Stay tuned!>

This article was contributed by HackerEarth and written by Shubham Jain, Developer, HackerEarth and Vivek Prakash, CTO & Co-founder, HackerEarth.